

XProc 3.0 - Strategies for merging documents

N o v e m b e r 1 6 , 2 0 2 0

E r i k S (i e a u t h o r s / e r i k - s i e g e l /)

Erik Siegel continues his series on the Xpressgoiffrog or documents.

Table of Contents

- Introduction
- The example case
- Using display as a template (recommended!)
- Wrapping all documents into
- Passing document as par
- Passing all documents as a collection
- Wrap up

Introduction

XPort is an XML-based language for documents in p
 conversions and ogeth e a s e t h e s u i l t s . o A d s u c a t o i o X P o r t t r t
 x m l . c o h t / t w p s w x m l . c s m t e c o i n n o t r u o y s t a i n l e
 (h t / t w p s w x m l . c t o i m c / l a e r s / 2 0 1 0 d d t f 0 0 5 c h - i 3 0 4 r d t a i n c l a e r a b o e t t
 p o t r s (h t / t w p s w x m l . c t o i m c / l a e r s / 2 0 2 0 d 0 3 0 2 8 d x p e c t i n g s . s o) v e p c s a -
 a l s o v X P o r t 3 t . h o e h s t / t x p s r : g b / r i t s e l e f o f o m m a t i e s v e n A n d w h t
 b o o k h t / t x p s r : s . n e t / p u b d i c a t i o n s o / a x t p o r t t l a n g u a g e !

This is a free export of the no bold cmu rwh ieng hparn d l i n g e x i m l s y e a d b c u
document so nien, t u s t i s g y a n o s X o r l e i n g t g e l a t s e l r a t' b e i t a d n o e d t h
other two and e a d s y m k e n s o w o s c n s e. O X P (a n d) o a t s i t i a v r i t l e . X S L

The example (<http://dist.hub.com/xcart/mugl-1e/xapmefv/>) is available at
To run the example helloworld.src.over available the test source code
found at (<http://www.gprocessors.html>)

The example case

To illustrate this, I found an example that was already working properly. <https://github.com/boxcarai/mugl-te/xapnp> (<https://github.com/boxcarai/mugl-te/xapnp>).

We're going to meet the **XML document merging**. It's a fancy way of saying the which solves the question: how do we combine multiple XML documents into a single XML document? XProc 3.0 pipeline?

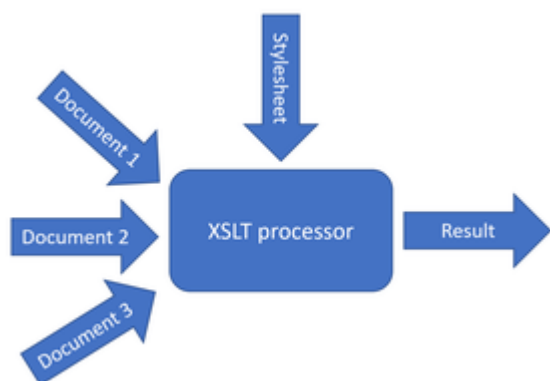


Figure 1: Merging three documents into one using XSLT

Now if it would be the **Figure 1** would be the easy designate one of the main input stylesheets other two as `log()` the function.

But what if there is no available command? you have a transformation and then first? Assume for some for one of the

and you need it. XML won't be used because it contains a processing instruction for use. So how do this using XProc? It's a simple, but it's a bit of a pain.

The three documents are merged into

1 An XML template that represents a single

Example 1. data / template Some HTML page template

```

<html>
  <head>
    <title>Temperature</title>
  </head>
  <body />
</html>
  
```

Our process is `body`.

2 A file with the `template.c` Any file that is a `cl` file is like

Example 2. data / temps / 1 Temperature entry for a single city

```

<citytemp id="12345" temp="20" />
  
```

All these entries are combined in a document. The master file `temps` includes

Example 3. data / temp: Temperature entries combined, using XInclude

```
<citytemps>
  <x i : i n c h u r e d t e m p s / 1 2 3 4 5 . x m l "
  <x i : i n c h u r e d t e m p s / 5 6 7 8 9 . x m l "
</citytemps>
```

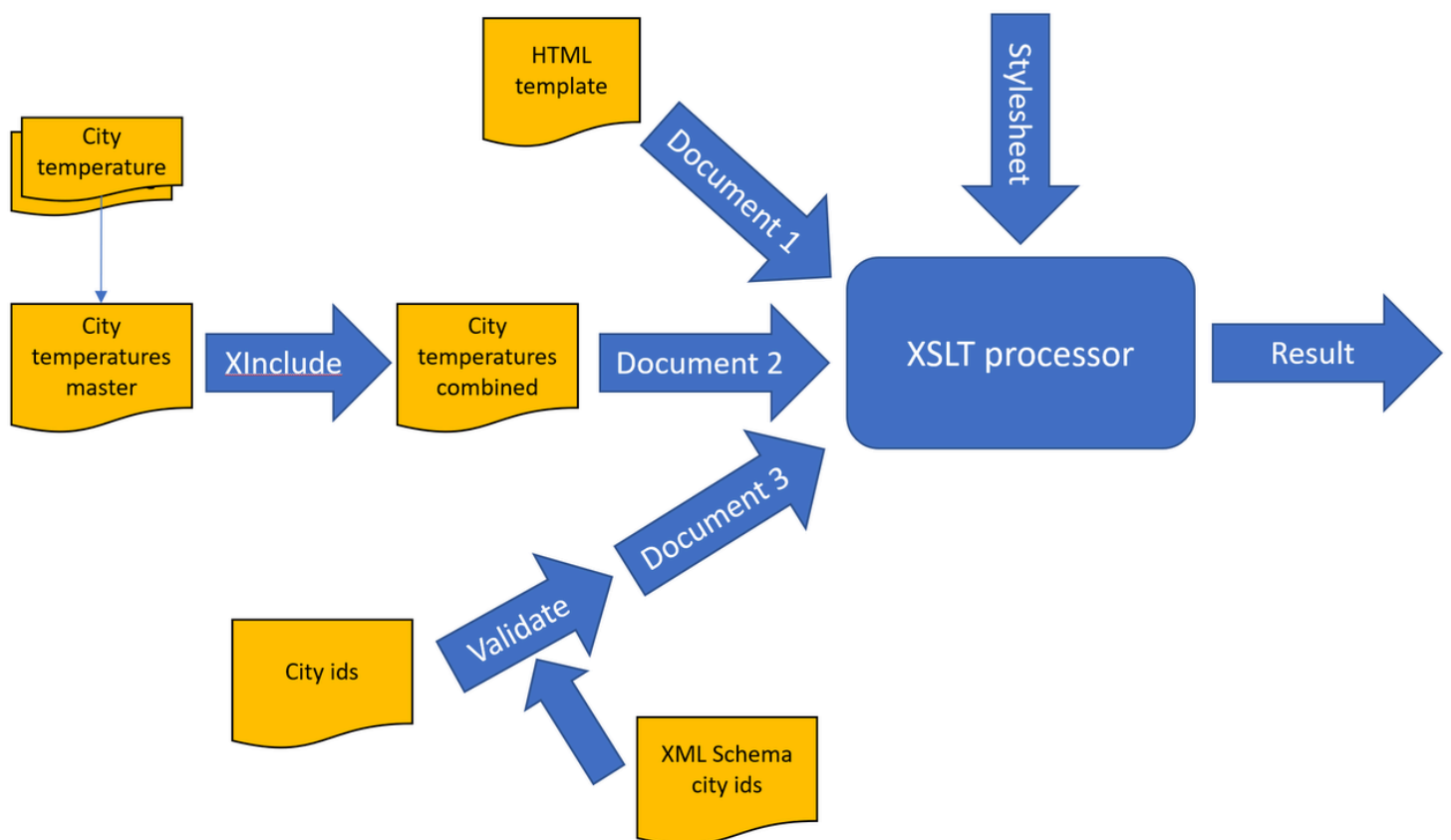
3 A document that links a society identifies the

Example 4. `data/city -i`: Temperature entry for a single city

```
< c i t y - i d s >
  < c i t y - i d s = 1 2 3 4 5 d' m ě A m s t e r / d a m "
  < c i t y - i d s = 5 6 7 8 9 d' m ě M o s c o w "
< / c i t y - i d s >
```

Now assume for this case that the development of the feedwater and the use of the feedwater is the same. The schema for the feedwater is as follows:

Also get the full of implementation is:



(/ m e d i a / i m e a - g 2 e . s w / i f d t u h r - 1 2 8 0 . p n g)

Figure 2: Example processing overview

The result will tell me a great deal about the nature of the problem, but it will not tell me how to solve it. However, it will tell me what the problem is (and in more complicated scenarios...

Using disk as temporary storage (not recommended!)

I found a new XProc and a more implemented pipeline using Ant script might be implemented in a test case. You can definitely XProc but definitely not recommended. Why would you consider a re-processor, in a way that it is a human wrote script code? And it also has a complicated pipeline. Not to mention that this can be done, just for the fun of it.

I actually designed a pipeline that is a simple script (and a change) expected some of its input do choice of a variant of all input. It is a simple pipeline, disk first.

Here is an XProc pipeline that is a simple pipeline that uses the template file as a main file and passes files as parameters:

Example 5. implementation - 1 / implementation Pipeline that uses temporary files for intermediate results

```
<p:declare-namespace xmlns:xp="http://www.w3.org/2001/XMLSchema"
```

```
;,, 0 , Cdbk qd sgd hmots onqs9 ,,
<p:input type="text" name="temp" value="" />
<p:input type="text" name="city" value="" />
<p:input type="text" name="ids" value="" />
```

```
;,, 1 , Cdbk qd sgd ntsots onqs9 ,,
<p:output type="text" name="temp" value="" />
<p:output type="text" name="city" value="" />
<p:output type="text" name="ids" value="" />
```

```
;,,
```

```
;,, 2 , Bqd sd sdlonq qx ehkd mc b sbg hsr m ld9 ,,
<p:file-create name="temp" value="" />
<p:file-read name="temp" value="" />
```

```
;,, 3 , Bnlotsd sgd etkk sdlodq stqd cnbtldms vqhsd hs sgd sdlonq qx ehkd
<p:include name="temp" value="" />
</p:include>
<p:store name="temp" value="" />
```

```
;,, 4 , U khc sd sgd bhsx,hcr cnbtldms- He sg hr hmu khc sgd ohodkhmd
<p:validate-with-schema name="temp" value="" />
<p:validate-with-schema name="temp" value="" />
</p:validate-with-schema>
```

```
;,, 5 , B sbg sgd m ld ne sgd bhsx,hcr cnbtldms9 ,,
<p:variable name="temp" value="" />
<p:variable name="temp" value="" />
```

```
;,, 6 , Cn sgd WR S oqnbdrhmf9 ,,
<p:xml
  parameter="temp" {
    'temp-filename': $temp-filename,
    'city-ids-filename': $city-ids-filename
  }
```

```

    <p:with-ipnpuet+template@main-/p>pipeline"
    <p:with-ipnpuet+stylesheet@main-/p>implementation"-1.xsl"
</p:xsl:t>

```

</p:declare-step>

1 The e eh ne eye sds o acume n p t a s s a eodu gihn itn lpsu:t p o r

- Since niott' clear why, chl icnpuet+template@main-/p> pipeline" nothing wv r v o t u h r i f n a g o n e r o f t h e o n t h e e r i s t a p r d i m a r k (primary=")true"

With the m o h a n o n e p r i m a r y p o f f a i l s s e t t h e d e f a u l d o m a t y p r i m a r y t r i b u t e .

- I was adteeldi y b e r s e i n d e t s a r s i p e v e y t h e i m p o r e c o u l d : sequence="false" content+itsy piess = "oxtm ly"t b u d t l i g i your pipe e b i b n e s t o m o v e i e f a c c i d e n t a l l y p a s s i n m u l t i d o c u m e n t , t h e s t e p w i l b t f a i l . I c o n s i d e r i t sequence="false" the d e f a u l d o m a t y s e e d q u t e h e a t t r i b u t

- To simplify run n d i h a g d d t h e m e p r i p e d o m e e a d t o i e d i n s t s w p e o r s h r e f a t t r i b u t s p e d i f f y s o m e o t h e r c o n n e t c s t i (o n n f t o h c o m m a n d l i n e f o r i n s t a n c e d) , t h i s d e f a u l t w i l

2 The outtpu its e d b e r a l s a r o n o m t i n a s t i d e f r e h c e e i p X M L e o (n t e n t - t y p e s = " x m l b u t a d v c i e s s o u t s t e m l p w h e n e s t l e e i a d d o z m e r d i s k e r (a l i z a t i o n = " m a p { ' m e t h o d o t f u n d t t e n l l y ' , f o c c r e s s o , u s u a l l y f o l l o w s a d v i c e . .

3 The p : f i l e - c r e a t e - t s e m p p t i d e a a y t e m l p e o r a n d o u t p u t s t i n a s i m g s a e t e n t e r e t e l e t e - o n - e x i t a t t r i b u t e c e s d o y s t o c o n v e n t l y d e l a e y t e f l t e h e a d t e e r s a r l g p s f n i s h e d .

After t h a y f e n e p a d e d c r w e p u t t e m p s - U R I l l e a i n m a t c h l e e .

4 The p : x i n c l u d e s p o e l s v t h e X i n c l u d e t s u r i f l t h e A f d r e p o r t h l a t e m p a y r f l e e a t e e d i p o u s l y

5 Then a w l e i d v a t e t h e c i t y i d e n t i f e r s w d o o w u n t e a m t e h o l e f n e

6 Since we want o o e u a r d s a t l y l l e a s d e i e t t i o m a d i s k o c u m e n t e s e d r i d e n t i f e r s d o o w e n e s r t t . c i l t h e y t - h i e d s - f i l e r i a n d e .

7 Final lay swer for the i.T.T. The i.m.g. i.X.S. lit mapusst for mat i ToML it se no pla
document, coming in g the i.e.p. l a p l e . W pass the f l e n a m e s f c
documents p s x h s s t o p a r a m e t o p s t i o n .

You might think that depth is the attribute that has the most influence on what is displayed. Is it?

Now look Example: at 5

- So the processor could extend, validate and store step in
opposite, a correlation exception would mean that when
a valid instruction is taken, the exception sheet kicks in a
condition is either a processor or the OS.

A s t y l e s h e a d i n g s t a n d a r d s (n o t i n t e n d e d t o b e a n e x a m p l e)

Example 6. implementation - 1 / implementation - 2

```

<xsl:stylesheet xmlns:xsl="http://www.w3.org/199
xmlns:xsl="http://www.w3.org/2001/XMLSchema"
exclude-result-prefixes="xsl" >

<xsl:mode name="text" output="text" >

<xsl:parameter name="temp-filename" type="string" >
<xsl:parameter name="city-ids-filename" type="string" >

<xsl:template match="/" >
  <xsl:copy
    <xsl:apply-templates select="/" />
    <xsl:for-each select="temp-filename" /> /* city te
      <xsl:variable name="city-id" select="xsl:string-length" >
        <p>doc(city-ids-filename) /* city-id[@id =
      </xsl:for-each>
    </xsl:copy>
  </xsl:template>

</xsl:stylesheet>

```

Wrapping all documents in to one

Using a transformer to construct an intermediate XML document called "Using a transformer to construct an intermediate XML document" not only a was a resoucers, it is also in an unnecessary. So how can we do better? A much simpler way to do this is to use a wrapper element to encompass all documents and stylesheets.

The wrapped document will look like

Example 7. The input documents wrapped in one

```

<wrapper-root-element>
  <html>
    ...
  </html>
  <citytemps>
    ...
  </citytemps>
  <city-ids>
    ...
  </city-ids>
</wrapper-root-element>

```

And here is a pipeline to do this:

Example 8. implementation - 2 / implementation Pipeline that wraps all documents into one as input for the XSLT transformation

```
< p : d e c l a r e v - e s r t s e i p b 3 = 0 n " a m ě m a i n - p i p e l i n e "
  x m l n s : " h t t p : / / w w w . w 3 . o r g / n s / x p r o c "
  x m l n s : " h t t p : / / w w w . w 3 . o r g / 2 0 0 1 / X M L S c h e m a "
```

```
< p : i n p u t r e m p l a t r e i m a r f a l s e d q u e n c e a l s e "
  c o n t e n t - t y p e l s t r e . / d a t a / t e m p l a t e . x m l "
```

```
<p:input type="text" value="content-type: application/xml" />
```

```
<p:input type="text" value="content - 'xml' file. / data / city - ids.xml"
```

```
<p:output type="text/html" data-bbox="111 100 888 150">
  content = "xpmels"
  serialized = {"method": "html", "indent": 4}
```

•
; "

```
;,, 0 , U khc sd sgd bhsx,hcr cnbtldms- He sg hr hmu khc sgd ohodkhmd  
<p : validate - with - x m l - s c h e m a > t e - c i t y - i d s - d o c u m e n t  
<p : with - ipnipputts e i t y - i d s @ m a i n - / p > i p e l i n e "  
    <p : with - ipnoprutts chenthar "e f = . / d a t a / x s d / c i t y > - i d s . x s d  
</ p : validate - with - x m l - s c h e m a >
```

```

;,, 1 , Bnlotsd sgd etkk sdldoq stqd cnbtloms ,,
<p:xinc h u a d e t e m p s - f i l e - x i n c l u d e d "
    <p:w i t h - i p n i p p u e t t e m p s @ m a i n - p / i > p e l i n e "
</p:xinclude>

```

```

;,, 2 , Bqd sd      rdptdmbd ne      kk mdbdrr qx cnbtdmsr      mc vq o sgdl hr
<p:wrap-sequence>
  <p:with-input
    pipe=
      template@main-pipeline
      result@temps-file-xincluded
      result@validate-city-ids-document
  />
</p:wrap-sequence>

```

```

;,, 3 , Cn sgd WR S oqnbdrhmf9 ,,
< p : x s l t >
  < p : w i t h - i p n o p r u t s t y l e s h e e d i m p l e m e n t a t i o n - 2 . x s l "
< / p : x s l t >

```

< / p : d e c l a r e - s t e p >

(For an explanation of the actions to be taken, see the appendix)
Example 5

1 We validate the city identifiers in the document.
When not using the validation step, the data is not validated and remains unchanged (that is, the data is not validated). (Please note that some of the data is not validated.)

2 The document is validated by the XSLT processor if the template

3 We create a new document from the template: the sequence of documents is processed and the sequence of documents is processed. Each document is processed.

- The template document is not processed.
- The template document is processed by the processor (the processor is the processor).
- The city identifiers are not processed by the processor (the processor is the processor).

Notice that the document is not processed by the processor (the processor is the processor).
city - identifiers are not processed by the processor (the processor is the processor).
an explicit dependency is not processed by the processor (the processor is the processor).

4 Now XSLT does its job: the document is processed by the processor (the processor is the processor).
primary source because of the implicit wrapper sequence.
processor steps.

Here is a simple XSLT sheet that appears as a document.

Example 9. implementation - 2 / implementation Stylesheet for Example 8 xsl

```

<xsl:stylesheet xmlns:xsl="http://www.w3.org/199
xmlns:xhtml="http://www.w3.org/2001/XMLSchema"
exclude-result-prefixes="xhtml" >

<xsl:mode name="html" output="xml" >

<xsl:template match="/" >
  <xsl:apply-templates />
</xsl:template>

<xsl:template match="body" >
  <xsl:copy
    <xsl:apply-templates mode="text" />
    <xsl:for-each select="/citytemps/citytemp" >
      <xsl:variable name="city-id" select="xsl:string-length(@id)" >
        <p> / * / city - ids / city - id [@id eq $city-id] / @r
      </xsl:for-each>
    </xsl:copy>
  </xsl:template>

</xsl:stylesheet>

```

Passing documents as parameters

Yet another example of passing documents as parameters is the `document()` function. In the following example, the `document()` function is used to pass a document to the `transform` function. The `document()` function is used to pass a document to the `transform` function. The `document()` function is used to pass a document to the `transform` function.

A pipeline that takes a document as input and produces a document as output.

Example 10. implementation - 3 / implementation Pipeline that uses document parameters to pass additional documents to the XSLT transformation

```
<p:declare-step ipb="main-pipeline"
  xmlns:tp://www.w3.org/ns/xproc"
  xmlns:xhst="tp://www.w3.org/2001/XMLSchema"
```

```
<p:input-template name="sequence"
  content="xpmels" file="/data/temp/ate.xml"
```

```
<p:input-template name="sequence"
  content="xpmels" file="/data/temp/s.xml"
```

```
<p:input-template name="city-ids"
  content="xpmels" file="/data/city-ids.xml"
```

```
<p:output-template name="result"
  content="xpmels"
  serialize="method": "html", "indent": true
```

```
;
```

```
0, U khc sd sg d bhsx,hcr cnbtdms- He sghr hr hmu khc sg d ohodkhmd
<p:validate-with-xslt-schema city-ids-document
  <p:with-ipnippets city-ids@main-/>pipeline"
  <p:with-ipnippets schema="file:/data/xsd/city-ids.xs
</p:validate-with-xml-schema>
```

```
1, Bnlotsd sg d etkk sd lodq stqd cnbtdms
<p:xinclude template="emps-file-xincluded"
  <p:with-ipnippets emps@main-pipeline"
</p:xinclude>
```

```
2, dedq sn sg d cnbtdmsr trhmf cnbtdms,mncd('sxd u qh akdr9
<p:variable name="city-temps-document" namespace="
<p:variable name="city-ids-document" namespace="
  pipeline-result@validate-city-ids-document"
```

```
3, Cn sg d WR S oqnbdr rhmf9
<p:xslt-parameterrasp={
  'city-temps-document': $city-temps-document
  'city-ids-document': $city-ids-document
}>"
  <p:with-ipnippets template@main-/>pipeline"
  <p:with-ipnippets stylesheet="implementations-3.xsl"
</p:xslt>
```

```
</p:declare-step>
```

(For an explanation of the actions to be taken, see the appendix)
Example 5

1 Like Example 1, the city identifier document.

2 The xsl:include element is used to include the template.

3 To be able to use the additional attributes, the following is required:

- The first template, xsl:template, is used to generate the output document. The xsl:include element is used to include the template.
- The second template, xsl:template, is used to generate the output document. The xsl:include element is used to include the template.

4 The xsl:template element is used to generate the output document. The xsl:include element is used to include the template.

And as a result, the following is the output:

Example 11. implementation - 3 / implementation - 3

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999
xmlns:xhtml="http://www.w3.org/2001/XMLSchema"
exclude-result-prefixes="xhtml" >

<xsl:mode name="xhtml" output="xml" />

<xsl:parameter name="city-temps-document" namespace="http://www.w3.org/1999
<xsl:parameter name="city-ids-document" namespace="http://www.w3.org/1999

<xsl:template name="copy" >
  <xsl:copy
    <xsl:apply-templates select="@*" />
    <xsl:for-each select="$city-temps-document/city-temp" >
      <xsl:variable name="city-id" select="$city-ids-document/city-id[@id=$city-temp/city-id]" />
      <p{$city-ids-document/city-ids/city-id[@id=$city-temp/city-id]}>
    </xsl:for-each>
  </xsl:copy>
</xsl:template>

</xsl:stylesheet>
```

Passing all documents as a collection

One way to do this is by passing all the documents as a single collection to the XSLT processor. This is done by using the `<xsl:apply-templates data-type="collection">` element.

- We're going to use the `<xsl:apply-templates data-type="collection">` element to pass a collection of documents to the XSLT processor. The `data-type="collection"` attribute is used to indicate that the documents are being passed as a collection.
- Setting the `data-type="collection"` attribute has the intended effect of making the `<xsl:apply-templates data-type="collection">` element behave like a `<xsl:apply-templates data-type="document">` element. This means that the XSLT processor will treat the collection of documents as a single document.
- We then pass the documents to the XSLT processor as a sequence (of type `document()` or `collection()`).
- With all this, the XSLT processor will still be able to process the input documents as a collection. The `<xsl:apply-templates data-type="collection">` element is used to pass a collection of documents to the XSLT processor.

Watch out: The `<xsl:apply-templates data-type="collection">` element is used to pass a collection of documents to the XSLT processor. The `<xsl:apply-templates data-type="document">` element is used to pass a single document to the XSLT processor. The `<xsl:apply-templates data-type="collection">` element is used to pass a collection of documents to the XSLT processor.

A pipeline that uses a collection to pass documents to the XSLT transformation

Example 12. implementation - 4 / implementation Pipeline that uses a collection to pass documents to the XSLT transformation

```
<p:declare-step xmlns:xst="http://www.w3.org/2001/XMLSchema"
  xmlns:tp="http://www.w3.org/ns/xproc"
  xmlns:xhst="http://www.w3.org/2001/XMLSchema"
```

```
<p:input-template name="first-sequence"
  content="xproc/1/data/temp/1.xml"
```

```
<p:input-template name="second-sequence"
  content="xproc/1/data/temp/2.xml"
```

```
<p:input-template name="city-ids"
  content="xproc/1/data/city-ids.xml"
```

```
<p:output-template name="result-sequence"
  content="xproc/1/serializable"
  serialize="method: 'html', indent: true"
```

```
;
```

```
0, Ukhcsgd bhsxhcr cnbtldms- He sghr hr hmu khcsgd ohodkhmd
<p:validate-with-xslt name="validate-city-ids" document="
  <p:with-input name="city-ids@main-/>pipeline"
  <p:with-input name="xsd/city-ids.xsd"
</p:validate-with-xslt-schema>
```

```
1, Bnlotsdsgd etkk sldodq stqd cnbtldms
<p:xinclude name="temps-file-xincluded"
  <p:with-input name="temps@main-pipeline"
</p:xinclude>
```

```
2, Cnsgd WR S oqnbdrhmf9
<p:xslt-template name="process-ss-documents"
  <p:with-input
    pipe=
      template@main-pipeline
      result@temps-file-xincluded
      result@validate-city-ids-document
  />
  <p:with-input name="stylesheet-implementation-4.xsl"
</p:xslt>
```

```
</p:declare-step>
```

(For an explanation of the [xproc](#) to [lang](#), see [output](#) or [examples](#) 5

1. [Example](#), [validate the city identifiers document](#).

2 By wrapping the documents in some container, we should be able to call `readp()` on all documents in

3 By passing the additional document and the set
called document as par

4 By passing the document as a document object to the
collection

I don't have a recommendation on which to use the app (I
documents (number 2) is also a good option, and I may be able to
XP or 1.0 pipelines and a good example of the XSLT plus the
(just a simple XML document) to the
having a run in a container. But the best option is to use the
the app if you are interested. You can also find the source code
on GitHub (https://github.com/erik-siegel/textualy).

So: experiment, pick the one you like. Happy

Article contents © 2020 Erik Siegel

© Textuality Services, Inc. except for the trademarks and the trademarks appearing on the site are the property of their
owners.